

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title

College Exam Hall Management System

Subject

Operating System (315319)

Program

Diploma in Computer Engineering

Academic Year

2026–2027

Project Type

OS Concept Simulation / Mini Project

Project Objective

To simulate OS-style resource management for a large college examination event by demonstrating **process management, multithreading, CPU scheduling, deadlock detection, memory management, and file management**. The project follows the five Course Outcomes (CO1–CO5) given in the OS project-based learning requirements.

1. Introduction

1.1 Purpose

The **College Exam Hall Management System** is a command-line based Operating System simulator designed to demonstrate how OS concepts can be applied to the management of a large examination event.

The system will simulate:

- Examination hall allocation
- Student exam sessions

- Invigilator assignment
- Process and thread management
- CPU scheduling
- Deadlock detection
- Memory page management
- Seating arrangement storage
- File allocation
- Department-wise directory management

The system will provide menu-driven interaction so that the user can select and observe different OS concepts.

The project follows the requirement that the simulator should demonstrate OS concepts in an interactive working system.

2. Problem Statement

Managing a large examination event involves handling a large number of students, examination halls, invigilators, seating arrangements and departmental records.

Manual management can result in:

- Improper hall allocation
- Unequal distribution of students
- Invigilator conflicts
- Resource conflicts
- Inefficient memory usage
- Difficulty in maintaining seating records

The proposed system simulates these activities using Operating System concepts such as **processes, threads, FCFS scheduling, Priority scheduling, deadlock detection, LRU page replacement and contiguous file allocation.**

3. Objectives

The main objectives of the system are:

1. To simulate an examination management system using OS concepts.
2. To classify the examination process as a **batch processing OS scenario.**
3. To represent each student's examination session as a **process.**
4. To represent an invigilator as a **thread** associated with examination activities.
5. To allocate examination halls using **FCFS scheduling.**
6. To assign invigilators using **Priority scheduling.**
7. To identify and detect possible hall-resource deadlocks.

8. To represent seating arrangements using memory pages.
 9. To use **LRU page replacement** when active hall data exceeds available memory.
 10. To store seating arrangement files using **contiguous file allocation**.
 11. To maintain a **tree-structured directory for each department**.
 12. To demonstrate all five OS Course Outcomes through one integrated simulator.
-

4. Scope of the System

The system will simulate the following activities:

4.1 Examination Management

- Add examination details.
- Add students.
- Add departments.
- Add examination halls.
- Display examination schedule.

4.2 Process Management

- Each student's examination session will be represented as a process.
- Process information such as PID, student ID, state and hall will be displayed.
- Process states can be represented as:
 - Ready
 - Running
 - Waiting
 - Completed

4.3 Thread Management

- Each invigilator will be represented as a thread.
- Threads will be associated with examination hall activities.
- Multiple invigilators can be simulated concurrently.

4.4 Hall Allocation

- Examination halls will be allocated using **FCFS**.
- Students/examination batches will be assigned according to their arrival/order.

4.5 Invigilator Allocation

- Invigilators will be assigned using **Priority Scheduling**.
- Higher-priority invigilators/tasks will receive preference.

4.6 Deadlock Detection

The system will simulate hall-resource conflicts and identify whether a deadlock condition exists.

4.7 Memory Management

- Seating plans will be divided into memory pages.
- Active hall data will be loaded into memory.
- When memory becomes full, **LRU** will determine which page should be replaced.

4.8 File Management

- Seating arrangements will be stored using **contiguous allocation**.
- A tree-structured directory will be maintained department-wise.

These requirements directly correspond to the project-specific CO mapping in the supplied PDF.

5. Course Outcome Mapping

CO	Requirement in Project	Implementation
CO1	Show system information and classify exam management	OS information + batch processing classification
CO2	Student session as process and invigilator as thread	Process and multithreading simulation
CO3	Hall allocation, invigilator scheduling and deadlock	FCFS + Priority + Deadlock Detection
CO4	Seating plan in memory pages and active hall management	Paging + LRU
CO5	Seating files and department directories	Contiguous allocation + Tree directory

The official OS COs are: OS services/components, process management, CPU scheduling, memory management and file management.

6. Functional Requirements

FR-01: System Information

The system shall display basic system information such as:

- Operating system type
- Hostname
- System version

- System uptime
- Number of active processes

The PDF specifically requires system information and classification of the examination system as a batch-processing scenario.

FR-02: Batch Processing Classification

The system shall classify the college examination management process as a:

Batch Processing OS Scenario

because multiple examination-related tasks can be processed as a group according to a predefined examination schedule.

FR-03: Student Process Creation

The system shall create a process for every student's examination session.

Each process shall contain information such as:

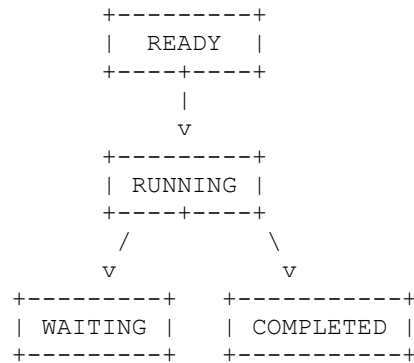
Field	Description
PID	Process ID
Student ID	Unique student identifier
Student Name	Name of student
Department	Student department
Exam	Examination name
Hall	Assigned hall
State	Current process state

Example:

```
PID: P101
Student ID: ST205
Student Name: Atharv
Department: Computer Engineering
Exam: Operating System
Hall: H-03
State: READY
```

7. Process States

The system shall simulate the following process states:



The process state will change according to the simulated examination activity.

8. Invigilator Thread Management

Each invigilator shall be represented as a **thread**.

An invigilator thread may contain:

- Thread ID
- Invigilator name
- Priority
- Assigned hall
- Current status

Example:

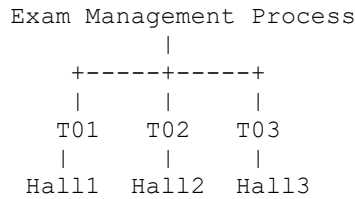
```
Thread ID : T01
Invigilator : Mr. Sharma
Priority : 1
Hall : H-01
Status : ACTIVE
```

The project requirement specifically defines the student's exam session as a process and the invigilator as a thread.

9. Multithreading Model

The system shall demonstrate that multiple invigilator threads can operate simultaneously.

Example:



This demonstrates the relationship between processes and threads.

10. Hall Allocation Using FCFS

The system shall allocate examination halls using the **First Come First Serve (FCFS)** algorithm.

Input

- Hall ID
- Hall capacity
- Examination batch
- Number of students

Processing

Halls will be allocated according to the order in which examination batches arrive.

Output

The system shall display:

- Hall allocation order
- Assigned hall
- Number of students
- Remaining capacity

Example:

FCFS HALL ALLOCATION

Batch	Hall	Students

CO5K-A	H-01	60
CO5K-B	H-02	55
CO5K-C	H-03	58

The project statement specifically requires **hall allocation using FCFS**.

11. Invigilator Assignment Using Priority Scheduling

The system shall assign invigilators using **Priority Scheduling**.

Each invigilator/task will have a priority value.

Example:

Invigilator Priority Hall

I01	1 H-01
I02	2 H-02
I03	3 H-03

A lower priority number can represent a higher priority.

The system shall display the order in which invigilators are assigned.

12. Hall Resource Deadlock Detection

The system shall simulate resource allocation between:

- Examination halls
- Invigilators
- Student batches
- Examination resources

The system shall identify whether a **deadlock condition** exists.

Example:

Hall H1 → Waiting for Invigilator I2

Invigilator I2 → Waiting for Hall H2

Hall H2 → Waiting for Invigilator I1

Invigilator I1 → Waiting for Hall H1

DEADLOCK DETECTED

The project statement specifically requires detection of hall-resource deadlock.

13. Memory Management

13.1 Seating Plan Paging

The seating arrangement of examination halls shall be divided into memory pages.

Example:

Hall H-01 Seating Plan

Page 1 → Seats 01-20

Page 2 → Seats 21-40

Page 3 → Seats 41-60

Page 4 → Seats 61-80

The system shall display:

- Page number
 - Hall number
 - Seat range
 - Page status
 - Last access information
-

14. LRU Page Replacement

When all available memory frames are occupied, the system shall use **Least Recently Used (LRU)** page replacement.

Example:

Memory Frames

Frame 1 → Page 2

Frame 2 → Page 5

Frame 3 → Page 1

If Page 3 is requested and memory is full:

Least Recently Used Page → Page 2

Page 2 → Removed

Page 3 → Loaded

The system shall display:

- Page reference sequence
- Memory frames
- Page hits

- Page faults
- Replaced pages

The project statement explicitly requires LRU for active hall data when all halls are occupied.

15. File Management

15.1 Seating Arrangement Files

Each examination seating arrangement shall be stored as a file.

Example:

```
CO5K/
├── Computer/
│   ├── OS_Exam.txt
│   ├── CN_Exam.txt
│   └── Java_Exam.txt
└── IT/
    ├── OS_Exam.txt
    └── DBMS_Exam.txt
```

16. Contiguous File Allocation

The system shall simulate **contiguous file allocation** for seating arrangement files.

Example:

```
File: OS_Exam.txt

Starting Block : 20
Length        : 5

20 → 21 → 22 → 23 → 24
```

The system shall display:

- File name
- Starting block
- File size
- Allocated blocks
- Allocation status

The project specifically requires seating arrangement files to use contiguous allocation.

17. Tree-Structured Directory

The system shall maintain a tree-structured directory according to departments.

Example:

```
College_Exam/
├── Computer_Engineering/
│   ├── OS/
│   │   ├── Seating.txt
│   │   └── Hall.txt
│   └── CN/
│       └── Seating.txt
├── Information_Technology/
│   └── OS/
│       └── Seating.txt
└── Electronics/
    └── OS/
        └── Seating.txt
```

The project requirement explicitly specifies a tree-structured directory per department.

18. Main Menu

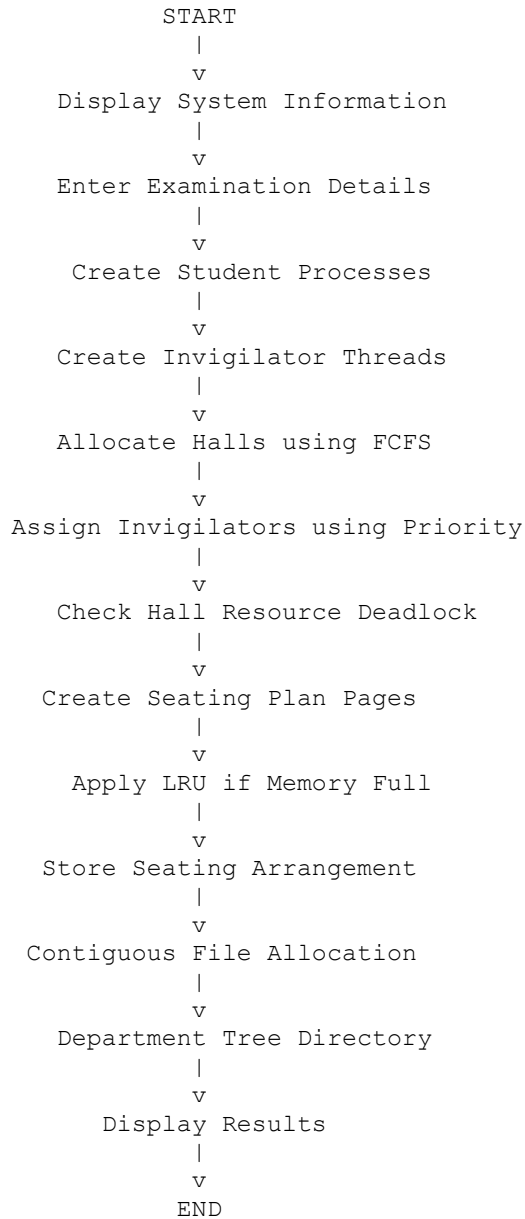
The proposed simulator shall use a menu-driven interface, consistent with the overall OS simulator requirement in the supplied document.

```
=====
COLLEGE EXAM HALL MANAGEMENT SYSTEM
=====
```

1. System Information
2. Student Process Management
3. Invigilator Thread Management
4. Hall Allocation - FCFS
5. Invigilator Allocation - Priority
6. Deadlock Detection
7. Seating Memory Management
8. LRU Page Replacement
9. File Management
10. Department Directory
11. Display Complete Exam Status
12. Exit

Enter Choice:

19. Overall System Flow



20. Non-Functional Requirements

20.1 Usability

- The system shall provide a simple menu-driven interface.
- Menu options shall be clearly displayed.
- Input and output shall be easy to understand.

20.2 Performance

- The simulator should process student and hall data efficiently.

- Scheduling algorithms should produce results without unnecessary delay.

20.3 Reliability

- Invalid input should be handled properly.
- The system should not terminate unexpectedly because of incorrect user input.

20.4 Accuracy

The implementation of FCFS, Priority, LRU and contiguous allocation must produce correct results.

The supplied project rubric gives significant importance to output correctness and algorithm accuracy.

20.5 Maintainability

- The program should be divided into logical modules.
- Each OS concept should have a separate function/module.
- Code should be easy to test and modify.

21. Hardware Requirements

Minimum recommended:

Component	Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB minimum
Storage	500 MB free space
Keyboard	Required
Display	Standard monitor

22. Software Requirements

Recommended Implementation

Programming Language: C or Python

The official project document allows **C or Python** for the working source code.

Development Environment

For Python:

- Python 3.x
- VS Code / PyCharm
- Windows/Linux

For C:

- GCC Compiler
- VS Code / Code::Blocks
- Windows/Linux

Optional OS Commands

Where required for system information:

- `uname`
- `hostname`
- `uptime`

The supplied OS project specification also uses OS/system commands such as `uname`, `uptime`, `hostname`, `ps`, `kill`, etc.

23. Data Requirements

The system will require the following major data.

Student Data

Student ID
Student Name
Department
Exam
Process ID
Hall ID
Seat Number
Process State

Hall Data

Hall ID
Hall Name
Capacity
Allocated Students
Available Seats
Hall Status

Invigilator Data

Invigilator ID
Name
Priority
Assigned Hall
Thread ID
Status

Seating Data

Department
Exam
Hall
Seat Number
Student ID
Page Number

File Data

File Name
Department
Starting Block
File Size
Allocated Blocks

24. Input Requirements

The system shall accept:

1. Number of students.
 2. Student details.
 3. Number of examination halls.
 4. Hall capacity.
 5. Number of invigilators.
 6. Invigilator priority.
 7. Examination details.
 8. Seating arrangement information.
 9. Memory frame/page information.
 10. File size and allocation information.
-

25. Output Requirements

The system shall display:

- System information
- Student processes
- Process states
- Invigilator threads
- Hall allocation

- FCFS execution order
- Priority scheduling order
- Deadlock status
- Memory page table
- LRU page replacement
- Page faults
- Seating arrangement
- File allocation blocks
- Department directory tree
- Final examination management status

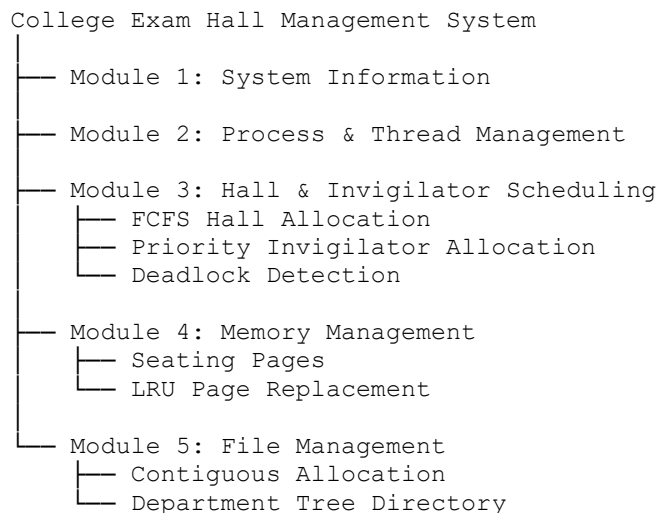
26. Constraints

1. The project will be a **simulation**, not an actual college examination management system.
2. It will not control real examination halls or physical resources.
3. Student and examination data will be simulated.
4. The system will demonstrate OS algorithms through program output.
5. The project should remain focused on the five specified COs.
6. The system should be implemented as a menu-driven program rather than an unnecessarily complex web application.

The official project specification describes the major project as an OS simulator and emphasizes building on the OS practicals already completed.

27. Module Structure

A clean implementation can be divided into these modules:



28. CO-wise Implementation

CO1 – Operating System

Implementation:

- Display system information.
- Display hostname.
- Display OS information.
- Display uptime.
- Explain examination management as a batch-processing scenario.

Expected Output:

```
OS INFORMATION
-----
OS Type       : Linux
Hostname      : Exam-System
System Uptime : 2 Hours
Active Tasks  : 25
```

```
Exam Management Type:
BATCH PROCESSING SYSTEM
```

CO2 – Process Management

Implementation:

- Create student processes.
- Assign PID.
- Display process states.
- Create invigilator threads.
- Demonstrate multiple threads.

The project requirement directly defines the student session as a process and invigilator as a thread.

CO3 – CPU Scheduling

Implementation:

- FCFS for hall allocation.
- Priority scheduling for invigilator assignment.
- Hall-resource deadlock detection.

This project-specific mapping differs from the generic simulator modules in the PDF, but still directly demonstrates CPU/process scheduling concepts required under CO3. The official practical list includes FCFS, SJF, Priority, Round Robin and Banker's Algorithm under CO3.

CO4 – Memory Management

Implementation:

- Divide seating arrangements into pages.
- Maintain memory frames.
- Track active hall data.
- Apply LRU page replacement.
- Display page faults.

The supplied project statement specifically maps seating pages and LRU to CO4.

CO5 – File Management

Implementation:

- Create seating arrangement files.
- Allocate files using contiguous allocation.
- Create department-wise tree directories.
- Display allocated blocks.

This directly follows the project statement's CO5 requirement.

29. Use Case Summary

Actor	Use Case
User	View system information
User	Add student
User	Create examination
User	Create hall
User	Create invigilator
User	Allocate hall
User	Assign invigilator
User	Detect deadlock
User	Manage seating pages

Actor	Use Case
User	Run LRU
User	Create seating file
User	View file allocation
User	View department directory
User	View final exam status

30. Expected System Output

At the end of execution, the system should provide an integrated view similar to:

```
=====
                        EXAM MANAGEMENT SUMMARY
=====

Total Students      : 120
Total Halls         : 3
Total Invigilators  : 6

Hall Allocation     : FCFS
Invigilator         : Priority

Deadlock Status     : SAFE

Memory Frames       : 4
Page Replacement    : LRU
Page Faults         : 7

File Allocation      : CONTIGUOUS

Directory Structure :
College_Exam
├── Computer
├── IT
└── Electronics

System Status       : EXAM MANAGEMENT COMPLETED
=====
```

31. Testing Requirements

The following test cases should be performed:

Test Case	Input	Expected Result
TC01	System information	OS information displayed
TC02	Add students	Student processes created
TC03	Add invigilators	Invigilator threads created
TC04	Multiple halls	FCFS hall allocation

Test Case	Input	Expected Result
TC05	Different priorities	Priority assignment
TC06	Circular resource dependency	Deadlock detected
TC07	Seating pages	Pages created
TC08	Full memory	LRU replacement
TC09	Seating file	Contiguous blocks allocated
TC10	Departments	Tree directory displayed

32. Security and Validation

The system should validate:

- Student ID should be unique.
 - Hall capacity should not be exceeded.
 - Invigilator should not be assigned to multiple halls simultaneously.
 - Invalid priority values should be rejected.
 - Invalid memory frame values should be rejected.
 - File size should be valid.
 - Duplicate file names within the same directory should be avoided.
-

33. Limitations

1. The system is a simulation and does not manage real college examinations.
 2. Real-time synchronization with college databases is not included.
 3. Actual physical hall monitoring is not included.
 4. Real student authentication is not required.
 5. The deadlock mechanism is simulated rather than controlling actual OS resources.
 6. The system focuses specifically on the OS concepts defined in the project statement.
-

34. Future Enhancements

The following features can be added later:

1. GUI-based examination management system.
2. Database integration.
3. Student login system.
4. Automated seating arrangement generation.
5. Real-time hall occupancy monitoring.
6. Web-based administrator dashboard.
7. Automatic timetable generation.

8. QR-based student attendance.
9. Automated invigilator scheduling.
10. Real-time notifications.

These are **future enhancements only**; they should not replace the core OS simulation requirements.

35. Project Deliverables

According to the supplied project document, the final submission should contain:

1. **Working source code**
2. **Sample output screenshots / terminal recordings**
3. **Project report**
4. **Presentation of 8–10 slides**
5. **Contribution log**
6. Live demonstration of the working project

These deliverables are explicitly listed in the project document.

36. Recommended Technology

For **this particular mini project**, I recommend:

Primary

Python 3

Why Python?

- Easy implementation of OS algorithms.
- Easy handling of lists, queues and dictionaries.
- Simple terminal output.
- Faster development for a mini project.
- Easy implementation of process/thread simulation.
- Suitable for demonstrating FCFS, Priority, LRU and file allocation.

Proposed Structure

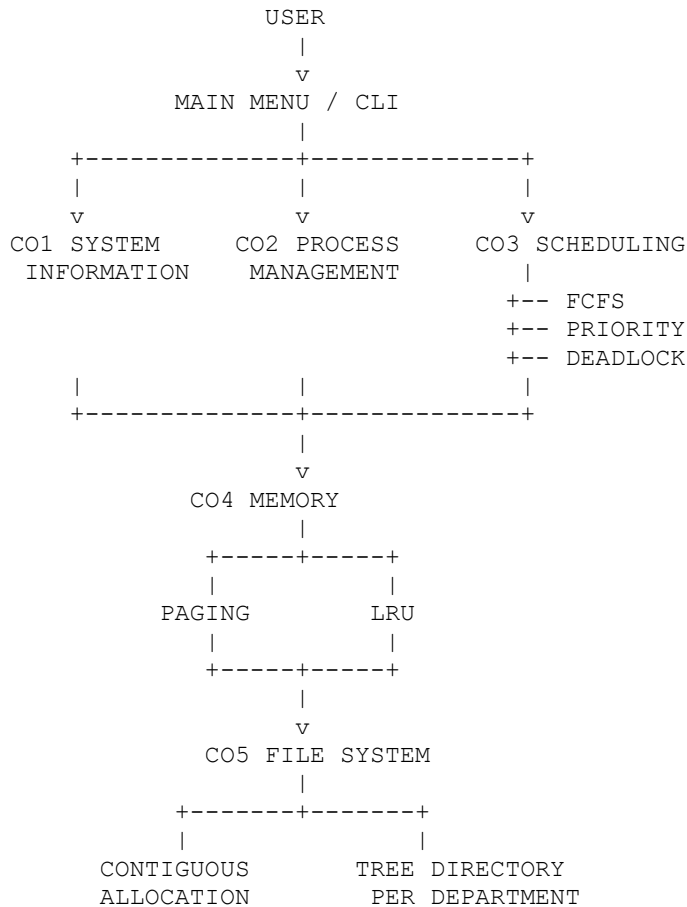
```
college_exam_os/  
├── main.py  
├── system_info.py  
├── process_manager.py  
└── thread_manager.py
```

```

├── hall_scheduler.py
├── deadlock.py
├── memory_manager.py
├── lru.py
├── file_manager.py
├── directory_manager.py
└── data/

```

37. Final System Architecture



38. Conclusion

The **College Exam Hall Management System** will provide a practical simulation of Operating System concepts in the context of a large college examination event. It integrates all five COs by implementing system information, student processes, invigilator threads, FCFS hall allocation, Priority-based invigilator assignment, deadlock detection, memory paging with LRU, contiguous file allocation and department-wise tree directories.

The design is intentionally kept as an **OS concept simulator**, matching the project structure specified in the supplied document rather than turning it into a generic college management

application. The official project framework also emphasizes connecting the simulator output with the OS practicals already completed by students.